Improving Classifier Performance on Imbalanced Software Defect Datasets Using Resampling Techniques

Talari Nandeesh and Ashu Mehta

Department of Computer Science and Engineering, Lovely Professional University, Phagwara, Punjab

E-mail : nandeeshtalari33@gmail.com, ashu.23631@lpu.co.in

2025 International Conference on Frontier Technologies and Solutions (ICFTS) | 979-8-3315-0276-8/25/$31.00 ©2025 IEEE | DOI: 10.1109/ICFTS62006.2025.11031775

Abstract- The accurate prediction of software defects is essential for enhancing detecting high-risk areas early in the development cycle to improve software quality. However, many software defect datasets suffer from class imbalance, where the performance of ML models suffers because the number of non-defective examples greatly exceeds the number of defective ones. This study explores the impact of various resampling methods SMOTE, ADASYN, and ROS in addressing this imbalance and improving defect prediction in two imbalanced datasets, PC2 and PC3. We assess the performance of five common machine learning classifiers, such as Random Forest, Decision Tree, and KNN Support Vector Classifier (SVC), and AdaBoost Classifier, under both normal and resampled conditions. Our findings demonstrate that resampling techniques notably improve the recall and F1-score for the defective class, resulting in more robust and accurate predictions. Among the resampling techniques, SMOTE and ADASYN consistently yielded better performance than ROS. The findings highlight the importance of addressing class disparity in the forecast of software defects and demonstrate the potential of resampling approaches in improving model performance, especially for minority classes. This works reviews and compares these techniques in detail and their use in practice in software engineering.

Keywords—software fault, Ml algorithms, sampling models, imbalance datasets.

I. INTRODUCTION

The prediction of fundamental issue in software engineering is software flaws, which can be found early in the program development process to greatly increase software quality. As a result of early defect detection helps developers to manage risks before they arise, early stage recognition contributes to the relatively lower costs of defect fixing ensuring the efficiency and reliability of software systems. Detection of software defects, however, poses a major challenge that is the common case of class imbalance in defect datasets. Such datasets tend to have a far higher proportion of non-defective software examples than flaws. and therefore the resulting data distributions are skewed.

This Due to this imbalance, performance of machine learning models may be negatively affected, as they prefer to classify with majority class (non-defective instances) and fail to achieve good performance on prediction of minority class (defective instances). This represents a significant barrier for proper defect identification, and this is particularly worrisome when the objective is to find infrequent but high-impact problems that can affect the proper functioning of a software or worse, its security.

Common machine learning algorithms employed regarding a software flaw prediction include Random forest, KNN, Support Vector Classifier (SVC), and AdaBoost. A common problem with general ML models with their training on imbalanced datasets is that they tend to get biased towards misclassifying defects and the F1-scores and recall for the minority class drop significantly. To counteract this problem, different methods (for example, SMOTE, ADASYN, and ROS) were proposed to manage the imbalance of the dataset either by making the dataset contain more defective examples or by making the dataset contain fewer non-defective instances. These resampling techniques are used to enhance the model's defect prediction capacity to the greater extent by resolving the class imbalance[19-21].

In this study, we will analyse the effectiveness of these resampling approaches besides the other effective resampling methods on the execution time and accuracy of the various classifiers of the machine learning algorithms in the software fault prediction process. Specifically, we evaluate the performance of 5 commonly used classifiers KNN, Random Forest, Decision Tree, SVC, and AdaBoost over 2 imbalanced defect datasets, PC2 and PC3. This paper discusses the performance of classifiers on imbalanced data and how applying different resampling techniques can improve the performance of each model, aimed at benefiting the relatively smaller defective class[15-18].

In software defect prediction (SDP), one main issue in the increasing domain is the class imbalance; specifically, correctly identifying the (positive class) faulty examples is

Authorized licensed use limited to: Lovely Professional University - Phagwara. Downloaded on June 25,2025 at 04:50:37 UTC from IEEE Xplore. Restrictions apply.

more critical than forecasting the majority (non-defective) class. Missed problems due to undetected defects can affect software performance, security, and user experiences. Thus, improving the defect prediction capabilities of the classifier, mainly with respect to underrepresented types of defects, is critical to deliver strong and reliable software[22-24].

II. LITREATURE REVIEW

1. Fault Prediction Using SVMs and GAs: Fault Prediction using SVM and Genetic Algorithms (GAs) Evaluation Use accuracy as a metric to evaluate the performance of the model. Analysis: Obtained from the PROMISE repository datasets, performing model (2011) [9].
2. Ensemble Methods for Fault Prediction: Ensemble Techniques: this study also tried out AdaBoostM1, Vote (StackingC). Performance Severity metrics of the model: recall, precision, accuracy and F-value Model Validation. Methodologies: K-fold cross-validation and ROC analysis (2015) [8][11][14].
3. Defective Software Module Detection: Data Preprocessing: Dealing with messy and incomplete datasets at preprocessing. Feature Extraction Techniques: experimenting with multiple extraction techniques (2018) [5].
4. SOA-Based Systems Reliability Enhancement: Feature Selection Strategy: One hot encoded prepared variable and various approaches implemented for feature selection (PCA, gain ratio, $\chi 2$ and correlation based approaches) (2019) [2].
5. Iterative Artificial Neural Network for Fault Prediction: What is actually included in this project is a prediction method. Performance Metrics. We define the concept of Employee Performance score using accuracy, recall, precision and F-measure metrics. Feature Selection strategy (2019) [6][12].
6. Software Fault Proneness Prediction with G-Lasso Regression: Prediction Models: Max G-Lasso, six ml models which predict Fault Prone, Take Group Lasso Regression (G-Lasso). Data Handling Techniques: SMOTE to solve imbalance data it may suggest to the recommendation, that the training data should be truly analysed. Dataset Assessment:to explore and understand how the variation of different datasets proposed may influence on the construction performance of the models (2019) [10].
7. Digital Communication Impact Study: Data Collection: Use of Datasets: Using datasets from the PROMISE repository. Analysis: Using VARX to assess the influence of digital communication on online engagement and tracking Google search trends Content analysis to determine brand personality traits. Performance Evaluation: Using metrics such as ACC, PD, and PF in assessing software project results (2020) [1].
8. Software Fault Prediction Using SVM and PSO: Prediction Approaches: One of the finest classifiers already exhibits superb performance in classification and prediction tasks. Feature Selection Approach: SVM based features selection. Optimization Techniques: Using GA and Particle Swarm Algorithm (2020) [3][12].
9. Software Fault Prediction with Optimization Algorithms and SVM: Prediction Models: SVM+PSO for fault prediction This is where the SVM is used for feature selection (2020) [4].
10. Software Fault Prediction with ML and DL Techniques: Learning Techniques: Using ML and DL algorithms for fault prediction245 Using 10 folds for cross-validation; I evaluated performance with ROC analysis. Feature selection approach: Use systematic algorithm selection and validation (2023) [7] [12].

TABLE I. REPRESENTS LITERATURE SURVEY

Author & Ref.no	**Year**	**Dataset**	**ML Algorithm**	**Evaluation Parameters**	**Result**
Di Martino et al[1]	2011	datasets extracted from the jEdit system, leveraging Chidamber and Kemerer (CK) metrics	Random Forest, Multi-Layer Perceptrons, Naïve Bayes, Decision Tree Algorithm C4.5, KNN, and Logistic Regression	F-Measure, Accuracy, Precision, and Recall	cumulative Accuracy of 80.14% and discussed possible challenges to the empirical investigation's validity, such as external validity, outcome validity, and validity of construction.
Hussain et al[2]	2015	Jedit-4.3 and Xalan-2.7	Naive Bayes, Logistic, J48, Voted Perceptron, and SMO	measurements of F-value, recall, accuracy, and precision	Jedit-4.3 97.76% free of bugs and Xalan-2.7 1.21% bugs free classes.
Raju et al[3]	2018	datasets KC1 and KC2.	SVM	accuracy, sensitivity, specificity, precision, and F-measure	99.54% -> JM1, 96.58% -> KC1 99.95% -> KC2 99.57% -> PC1 all had classifier accuracy levels

Authorized licensed use limited to: Lovely Professional University - Phagwara. Downloaded on June 25,2025 at 04:50:37 UTC from IEEE Xplore. Restrictions apply.

					above 97%.
Bhandari et al[4]	2019	Data from eBay web services and OWLS-TC4	Naïve Bayes, ANN, AdaBoost, Decision Tree, Random Forest, SVM	PCA, gain ratio, χ2, and correlation-based selection. ROC values, recall, precision, accuracy, and f-measure	using source code metrics and ML techniques effectively predicts faults, with the J48 decision tree algorithm performing best.
Turabieh et al[5]	2019	xalan-2.7, log4j-1.1	ANN, neural network, BGA, BPSO, or BACO	Future selection	categorisation rate, which is 0.8358 on average across all datasets
Goseva et al[6]	2019	22 datasets taken from projects that are open-source	Naive Bayes,RF	Together with statistical tests, recall, precision, FPR, F-score, G-score, and AUC	Use RF on balanced datasets, and it did better than the other students.
Yucalar et al[7]	2020	firm's LinkedIn page over 104 weeks, 15 datasets from PROMISE	Naïve Bayes, SVM	ACC, ROC curve, AUC, PD, PF, and FM	Ensemble predictors like In order to improve fault prediction.
Alsghaier et al[8]	2020	used in 24 datasets Twelve NASA MDP datasets and twelve open-source Java applications	NaiveBayes, J48, and decision tree in all tested datasets. SVM, GA, and PSO	F-measure, recall, specificity, precision, SD, error rate, and correctness	accuracy for fault prediction of up to 90%. Seven of the twelve datasets have an accuracy range between 0.90 and 1.00, whereas the range of the error rate is 0.004 to 0.329. The range of accuracy is 68.8% to 99.5%.
Sharma et al[9]	2020	validated on benchmark datasets	Support Vector Machines	include the F-measure, standard deviation, recall, accuracy, specificity, error rate, and precision.	To tackle broader classification problems beyond the of software engineering
Borandag et al[10]	2023	three datasets - Eclipse, SFP XP, and Apache Active MQ-TDD	ML algorithms, deep learning (CNN, LSTM, BiLSTM), and RNNBDL	ACC, AUC, FM, and KE	classifiers is 94.30%, whereas 93.61% is the average ACC. The Diff value, which displays the variation between the two main indicators, points to a 0.69% gain.

III. METHODOLOGY

A. Data Collection

For this study, three widely recognized benchmark datasets were utilized: PC2 and PC3 Numerous software indicators, including code complexity, churn, and other characteristics, are included in these databases and can be used to assess the likelihood of a software module being flawed. Finding these high-risk modules early in the building process is the aim of defect prediction, which can greatly save maintenance costs and enhance software quality. However, a significant challenge in this research is the issue of class imbalance where most of the instances in the dataset belong to the non-defective class. This imbalance makes it harder for predictive models to correctly identify the minority class (defective modules), and it necessitates the use of specialized techniques to mitigate this issue and improve model performance.

Authorized licensed use limited to: Lovely Professional University - Phagwara. Downloaded on June 25,2025 at 04:50:37 UTC from IEEE Xplore. Restrictions apply.

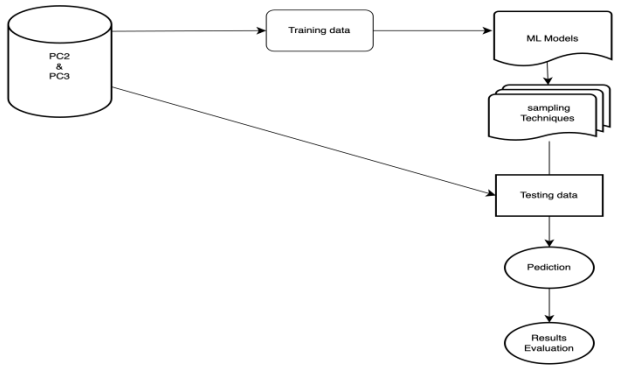

Fig. 1. Represents the flow of process.

B. Data Preprocessing

1. **Handling Missing Values:** Missing or incomplete data is a frequent occurrence in real world datasets. In this case the datasets had rows without value that were dropped. This hidden set thus avoids overfitting and biasing the networks and models, and keeps the training undisclosed from the testing. Other methods such as imputation could have been used, however it was decided that, for the purposes of this study, removing the rows was more consistent and simple.
2. **Binary Label Transformation:** We converted the target variable (defect status of software modules) into a binary format whereby defect-prone modules were given a value of "1" and non-defective modules were coded as "0." For binary classification models, this binary transformation was imperative, and thus all datasets are possessed similar in terms of target variable format.
3. **Train-Test Split:** In order to fairly gauge model performance, The datasets were divided into subgroups for testing and training. 80% of the information is used as a training set for the models, and the remaining 20% is set aside for testing. By doing so, we make the sure that the models are assessed on the new unseen data, which allows us to get a more honest measure of how these models generalize.

C. Resampling Techniques

Due to the class imbalance problem, resampling techniques were applied to balance the class distribution in the training datasets. The three techniques used in this research are:

1. **SMOTE:** SMOTE interpolates between existing instances to create new synthetic instances for the minority class (defective modules). By producing fresh samples rather than merely replicating the minority class instances, which can result in overfitting, this method aids in addressing class imbalance.
2. **ADASYN (Adaptive Synthetic Sampling):** ADASYN extends SMOTE by focusing on creating synthetic samples in areas where the minority class is harder to classify—specifically near the decision boundary. ADASYN identifies difficult-to-classify instances in the minority class and generates synthetic samples around those instances.
3. **Random Oversampling:** This technique is a simpler form of resampling that involves duplicating instances from the minority class until both classes are balanced. Random oversampling can be useful in addressing class imbalance, but it can also lead to concerns with overfitting because no new information is added, only duplicates of the minority class.

D. Model Selection

1. **K-Nearest Neighbors (KNN):** One more simple but quite effective classification algorithm that checks the predominant class in the data point's closest neighbors. You are not aware of anything previously learned, except for all the class labels that you learned at once, and force you to classify them.
2. **Random Forest:** A type of ensemble learning where you build many decision trees and combine their output.
3. **Support Vector Machines (SVM):** Finding the hyperplane that best splits a dataset into two classes is the goal of this discriminative classifier. It functions effectively in high-dimensional spaces, also in high dimensional space where classes are separated with larger distance.
4. **Decision Trees:** Associate Data into a Tree A tree based algorithm that partitions the information recursively primarily based on characteristic values to create a tree-like construction. It is highly interpretable and helpful for understanding feature importance.
5. **AdaBoost:** It is an ensemble method which builds weak classifiers (most often decision trees) one at a time. Each new model refines and corrects the errors of the preceding one, and they are very effective at improving predictive accuracy.

E. Performance Metrics

The following performance metrics were used to evaluate how well the models performed:

1. **Accuracy:** This is calculated by dividing the total number of cases in the dataset by the number of accurate predictions. Despite being one of the most often used measures, accuracy it can be deceptive when the dataset is heavily unbalanced, in these cases models can achieve high accuracy by only predicting the majority class.
2. **Precision:** This becomes especially crucial when false positives are expensive, as in software defect prediction, which incorrectly identifying a non-defective module as defective may entail unnecessary effort and cost.
3. **Recall:** Recall is the proportion of actual defect-prone modules that were correctly identified. In this context it's important to recall that, for defect

Authorized licensed use limited to: Lovely Professional University - Phagwara. Downloaded on June 25,2025 at 04:50:37 UTC from IEEE Xplore. Restrictions apply.

prediction, one might like to capture as many defective modules as possible, even at the cost of a higher false positive rate.

4. **F1-Score:** The one metric that balances precision and recall is the harmonic mean of the two. When working with unbalanced datasets, when no precision nor recall can be disregarded, it is especially helpful.

F. Evaluation Strategy

Train and Evaluate Models: Each classifier was trained on the resampled datasets (SMOTE, ADASYN, Random Oversampling) and tested on the separate test set.

IV. Results

TABLE II. Represent the results of five machine learning models of Pc2 datasets.

PC2 Dataset

ML Model	Precision (0)	Precision (1)	Recall (0)	Recall (1)	F1-Score (0)	F1-Score (1)	Accuracy	Macro Avg Precision	Macro Avg Recall	Macro Avg F1-Score
KNeighborsClassifier (Normal)	0.97	0	1	0	0.99	0	0.97	0.49	0.50	0.49
RandomForestClassifier (Normal)	0.97	0	1	0	0.99	0	0.97	0.49	0.50	0.49
DecisionTreeClassifier (Normal)	0.98	0.50	0.99	0.25	0.99	0.33	0.97	0.74	0.62	0.66
SVC (Normal)	0.97	0	1	0	0.99	0	0.97	0.49	0.50	0.49
AdaBoostClassifier (Normal)	0.97	0	0.99	0	0.98	0	0.97	0.49	0.50	0.49
KNeighborsClassifier (SMOTE)	0.98	0.04	0.63	0.50	0.77	0.07	0.63	0.51	0.57	0.42
RandomForestClassifier (SMOTE)	0.98	0.50	0.99	0.25	0.99	0.33	0.97	0.74	0.62	0.66
DecisionTreeClassifier (SMOTE)	1.00	0.21	0.90	1.00	0.95	0.35	0.90	0.61	0.95	0.65
SVC (SMOTE)	0.97	0.03	0.79	0.25	0.87	0.06	0.78	0.50	0.52	0.47
AdaBoostClassifier (SMOTE)	0.99	0.33	0.97	0.50	0.98	0.40	0.96	0.66	0.74	0.69
KNeighborsClassifier (ADASYN)	0.98	0.04	0.66	0.50	0.79	0.07	0.65	0.51	0.58	0.43
RandomForestClassifier (ADASYN)	0.98	0.50	0.99	0.25	0.99	0.33	0.97	0.74	0.62	0.66
DecisionTreeClassifier (ADASYN)	1.00	0.21	0.90	1.00	0.95	0.35	0.90	0.61	0.95	0.65
SVC (ADASYN)	0.99	0.09	0.79	0.75	0.88	0.16	0.79	0.54	0.77	0.52
AdaBoostClassifier (ADASYN)	0.98	0.17	0.97	0.25	0.97	0.20	0.95	0.57	0.61	0.59
KNeighborsClassifier (ROS)	0.97	0	0.94	0	0.95	0	0.91	0.49	0.47	0.48
RandomForestClassifier (ROS)	0.97	0	1	0	0.99	0	0.97	0.49	0.50	0.49
DecisionTreeClassifier (ROS)	0.98	0.09	0.86	0.50	0.92	0.15	0.85	0.54	0.68	0.54
SVC (ROS)	0.98	0.04	0.82	0.25	0.89	0.06	0.81	0.51	0.54	0.48
AdaBoostClassifier (ROS)	0.98	0.25	0.98	0.25	0.98	0.25	0.96	0.61	0.61	0.61

TABLE III. Represent the results of five machine learning models of Pc3 datasets.

PC3 Dataset

ML Model	Precision (0)	Precision (1)	Recall (0)	Recall (1)	F1-Score (0)	F1-Score (1)	Accuracy	Macro Avg Precision	Macro Avg Recall	Macro Avg F1-Score
KNeighborsClassifier (Normal)	0.84	0.4	0.98	0.06	0.91	0.1	0.83	0.62	0.52	0.50

Authorized licensed use limited to: Lovely Professional University - Phagwara. Downloaded on June 25,2025 at 04:50:37 UTC from IEEE Xplore. Restrictions apply.

RandomForestClassifier (Normal)	0.85	0.75	0.99	0.09	0.92	0.15	0.85	0.80	0.54	0.53
DecisionTreeClassifier (Normal)	0.83	0	0.97	0	0.90	0	0.81	0.42	0.48	0.45
SVC (Normal)	0.84	0	1	0	0.91	0	0.84	0.42	0.50	0.46
AdaBoostClassifier (Normal)	0.85	0.36	0.96	0.11	0.90	0.17	0.82	0.61	0.54	0.54
KNeighborsClassifier (SMOTE)	0.91	0.31	0.75	0.6	0.82	0.41	0.72	0.61	0.67	0.61
RandomForestClassifier (SMOTE)	0.90	0.57	0.93	0.49	0.92	0.52	0.86	0.73	0.71	0.72
DecisionTreeClassifier (SMOTE)	0.88	0.33	0.83	0.43	0.86	0.38	0.77	0.61	0.63	0.62
SVC (SMOTE)	0.89	0.4	0.87	0.46	0.88	0.43	0.80	0.65	0.66	0.65
AdaBoostClassifier (SMOTE)	0.87	0.33	0.88	0.31	0.87	0.32	0.79	0.60	0.60	0.60
KNeighborsClassifier (ADASYN)	0.90	0.27	0.71	0.57	0.79	0.37	0.69	0.58	0.64	0.58
RandomForestClassifier (ADASYN)	0.89	0.50	0.92	0.43	0.90	0.46	0.84	0.70	0.67	0.68
DecisionTreeClassifier (ADASYN)	0.90	0.34	0.81	0.51	0.85	0.41	0.76	0.62	0.66	0.63
SVC (ADASYN)	0.90	0.41	0.86	0.51	0.88	0.46	0.80	0.66	0.69	0.67
AdaBoostClassifier (ADASYN)	0.89	0.45	0.91	0.40	0.90	0.42	0.82	0.67	0.65	0.66
KNeighborsClassifier (ROS)	0.89	0.29	0.77	0.49	0.82	0.36	0.72	0.59	0.63	0.59
RandomForestClassifier (ROS)	0.86	0.47	0.96	0.20	0.91	0.28	0.83	0.66	0.58	0.59
DecisionTreeClassifier (ROS)	0.94	0.30	0.65	0.80	0.77	0.44	0.67	0.62	0.72	0.60
SVC (ROS)	0.90	0.40	0.86	0.49	0.88	0.44	0.80	0.65	0.67	0.66
AdaBoostClassifier (ROS)	0.89	0.37	0.84	0.49	0.87	0.42	0.78	0.63	0.66	0.64

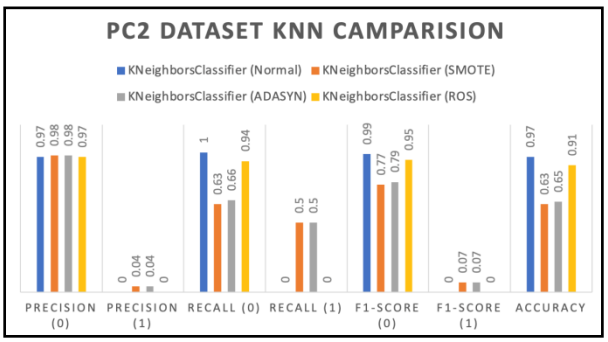

Fig. 2: Represents comparison of KNN of PC2 dataset.

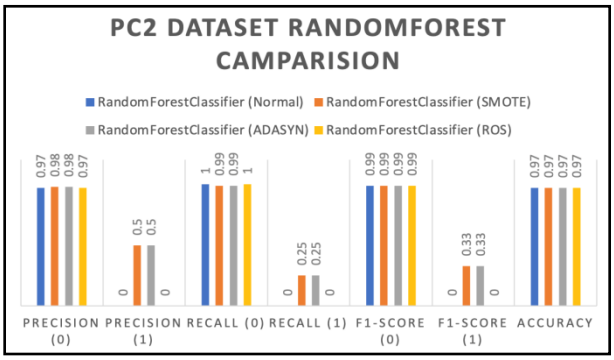

Fig. 3: Represents comparison of Random Forest of PC2 dataset.

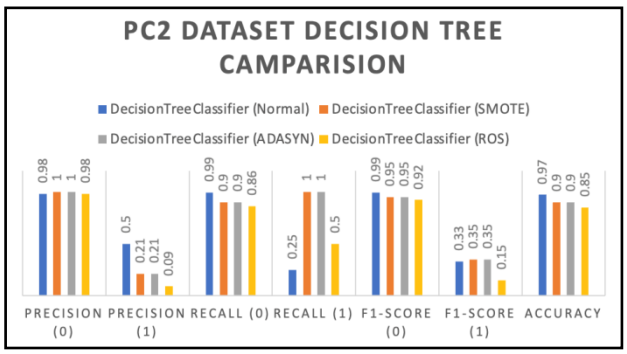

Fig. 4: Represents comparison of Decision Tree of PC2 dataset.

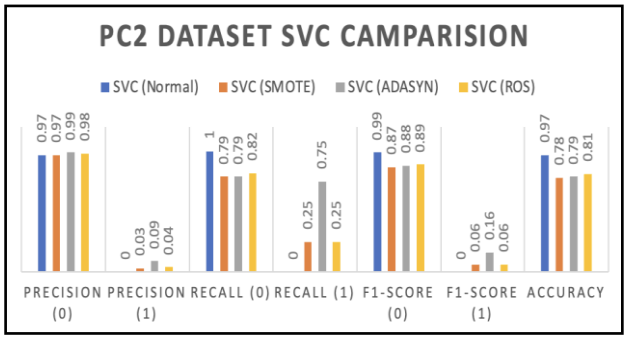

Fig. 5: Represents comparison of SVC of PC2 dataset.

Authorized licensed use limited to: Lovely Professional University - Phagwara. Downloaded on June 25,2025 at 04:50:37 UTC from IEEE Xplore. Restrictions apply.

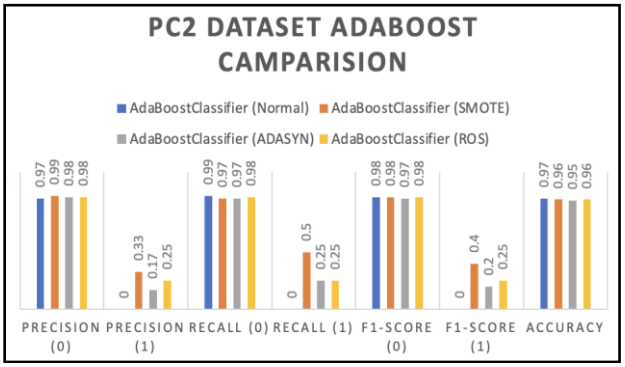

Fig. 6: Represents comparison of Adaboost of PC2 dataset.

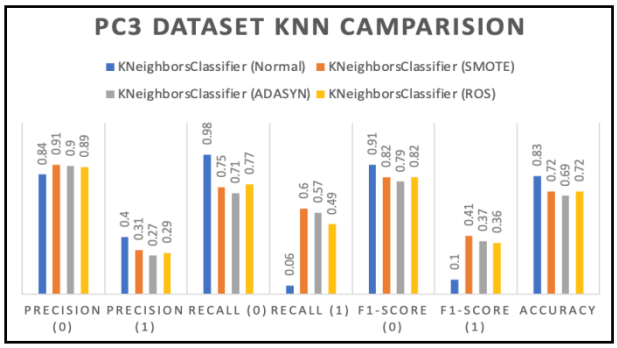

Fig. 7: Represents comparison of KNN of PC3 dataset.

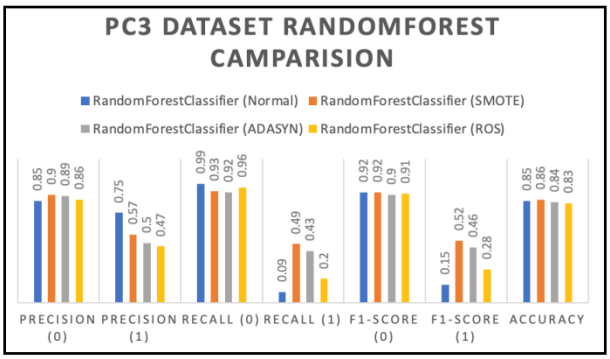

Fig. 8: Represents comparison of Random Forest of PC3 dataset.

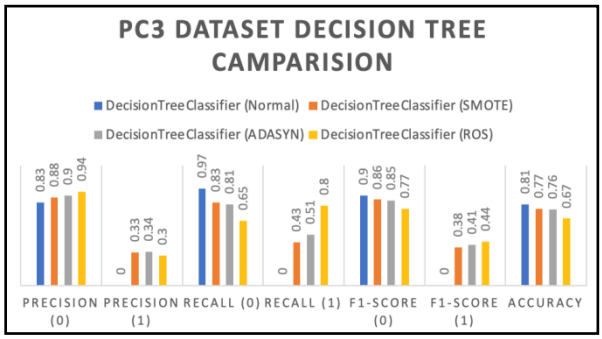

Fig. 9: Represents comparison of Decision Tree of PC3 dataset.

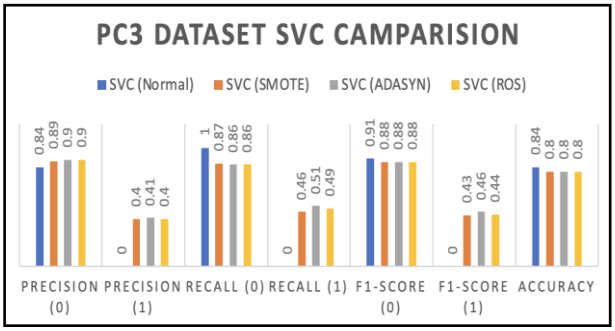

Fig. 10: Represents comparison of SVC of PC3 dataset.

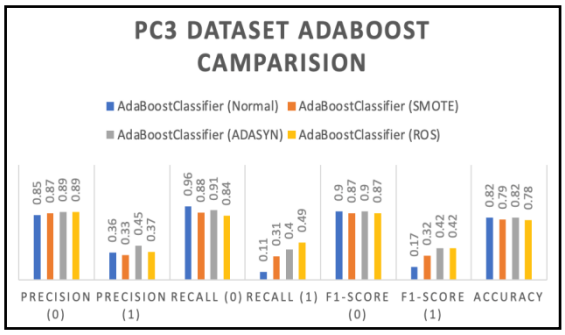

Fig. 11: Represents comparison of Adaboost of PC3 dataset.

V. CONCLUSION

Specific class imbalance handling methods can significantly boost the software defect prediction performance. SMOTE, ADASYN and ROS resampling techniques utilize in improving machine learning model performance, recall and F1-score improved significantly in defective instances (minority class). Based on the resampling strategies, both SMOTE and ADASYN outperformed generally ROS with higher prediction accuracy, improving the imbalanced problem posed in the datasets.

As observed from the comparison, out of the five of the popular ML classifiers KNN, Decision Tree, Random forest, SVC, and AdaBoost, although they all benefitted from the use of resampling, the magnitude of performance changed based on the technique used and the characteristics of the dataset when performing a return on the investment. These experimental findings emphasize the importance of choosing the appropriate resampling method to enhance defect prediction models.

REFERENCES

[1]. Yucalar, F., Ozcift, A., Borandag, E., & Kilinc, D. (2020). Multiple-classifiers in software quality engineering: Combining predictors to improve software fault prediction ability. Engineering Science and Technology, an International Journal, 23(4), 938-950.

[2]. Bhandari, G. P., Gupta, R., & Upadhyay, S. K. (2019). An approach for fault prediction in SOA-based systems using machine learning techniques. Data Technologies and

Authorized licensed use limited to: Lovely Professional University - Phagwara. Downloaded on June 25,2025 at 04:50:37 UTC from IEEE Xplore. Restrictions apply.

Applications, 53(4), 397-421.
[3]. Alsghaier, H., & Akour, M. (2020). Software fault prediction using particle swarm algorithm with genetic algorithm and support vector machine classifier. Software: Practice and Experience, 50(4), 407-427.
[4]. Sharma, D., & Chandra, P. (2020). Linear regression with factor analysis in fault prediction of software. Journal of Interdisciplinary Mathematics, 23(1), 11-19.
[5]. Raju, K. S., Murty, M. R., Rao, M. V., & Satapathy, S. C. (2018). Support vector machine with k-fold cross validation model for software fault prediction. International Journal of Pure and Applied Mathematics, 118(20), 321-334.
[6]. Turabieh, H., Mafarja, M., & Li, X. (2019). Iterated feature selection algorithms with layered recurrent neural network for software fault prediction. Expert systems with applications, 122, 27-42.
[7]. Borandag, E. (2023). Software fault prediction using an RNN-based deep learning approach and ensemble machine learning techniques. Applied Sciences, 13(3), 1639.
[8]. Hussain, S., Keung, J., Khan, A. A., & Bennin, K. E. (2015, September). Performance evaluation of ensemble methods for software fault prediction: An experiment. In Proceedings of the ASWEC 2015 24th Australasian software engineering conference (pp. 91-95).
[9]. Di Martino, S., Ferrucci, F., Gravino, C., & Sarro, F. (2011). A genetic algorithm to configure support vector machines for predicting fault-prone components. In Product-Focused Software Process Improvement: 12th International Conference, PROFES 2011, Torre Canne, Italy, June 20-22, 2011. Proceedings 12 (pp. 247-261). Springer Berlin Heidelberg.
[10]. Goseva-Popstojanova, K., Ahmad, M., & Alshehri, Y. (2019, July). Software fault proneness prediction with group lasso regression: On factors that affect classification performance. In 2019 IEEE 43rd Annual Computer Software and Applications Conference (COMPSAC) (Vol. 2, pp. 336-343). IEEE.
[11]. Mehta, A., Kaur, N., & Kaur, A. (2025). An Ensemble Voting classification Approach for Software defects prediction. International Journal of Information Technology, 1-8.
[12]. Pandey, Shivam Umesh, and Ashu Mehta. "Exploring the Effectiveness of Software Fault Detection Algorithms Through Machine Learning Techniques: A Comprehensive Review." Available at SSRN 4483839 (2023).
[13]. Kulamala, V. K., Kumar, L., & Mohapatra, D. P. (2021). Software fault prediction using LSSVM with different kernel functions. Arabian Journal for Science and Engineering, 46(9), 8655-8664.
[14]. Scanniello, G., Gravino, C., Marcus, A., & Menzies, T. (2013, November). Class level fault prediction using software clustering. In 2013 28th IEEE/ACM International Conference on Automated Software Engineering (ASE) (pp. 640-645). IEEE.
[15]. Mehta, Ashu, Amandeep Kaur, and Navdeep Kaur. "Optimizing Software Fault Prediction using Voting Ensembles in Class Imbalance Scenarios." International Journal of Performability Engineering 20, no. 11 (2024).
[16]. Hassouneh, Y., Turabieh, H., Thaher, T., Tumar, I., Chantar, H., & Too, J. (2021). Boosted whale optimization algorithm with natural selection operators for software fault prediction. IEEE Access, 9, 14239-14258.
[17]. Joon, A., Tyagi, R. K., & Kumar, K. (2020, June). Noise filtering and imbalance class distribution removal for optimizing software fault prediction using best software metrics suite. In 2020 5th international conference on communication and electronics systems (ICCES) (pp. 1381-1389). IEEE.
[18]. Alweshah, M., Kassaymeh, S., Alkhalaileh, S., Almseidin, M., & Altarawni, I. (2023). An efficient hybrid mine blast algorithm for tackling software fault prediction problem. Neural Processing Letters, 55(8), 10925-10950.
[19]. Wahono, R. S., Suryana, N., & Ahmad, S. (2014). Metaheuristic optimization based feature selection for software defect prediction. J. Softw., 9(5), 1324-1333.
[20]. Xu, Z., Liu, J., Yang, Z., An, G., & Jia, X. (2016, October). The impact of feature selection on defect prediction performance: An empirical comparison. In 2016 IEEE 27th international symposium on software reliability engineering (ISSRE) (pp. 309-320). IEEE.
[21]. Zhou, Y., & Leung, H. (2006). Empirical analysis of object-oriented design metrics for predicting high and low severity faults. IEEE Transactions on software engineering, 32(10), 771-789.
[22]. Liu, Y., Khoshgoftaar, T. M., & Seliya, N. (2010). Evolutionary optimization of software quality modeling with multiple repositories. IEEE Transactions on Software Engineering, 36(6), 852-864.
[23]. Menzies, T., Greenwald, J., & Frank, A. (2006). Data mining static code attributes to learn defect predictors. IEEE transactions on software engineering, 33(1), 2-13.
[24]. Yu, Q., Qian, J., Jiang, S., Wu, Z., & Zhang, G. (2019). An empirical study on the effectiveness of feature selection for cross-project defect prediction. IEEE Access, 7, 35710-35718.

Authorized licensed use limited to: Lovely Professional University - Phagwara. Downloaded on June 25,2025 at 04:50:37 UTC from IEEE Xplore. Restrictions apply.